

Helix OTA Firebase Deployment

Helix OTA — Firebase Deployment Plan + Git-Ignored Secrets / Fresh-Clone Manifest

Revision: 1

Last modified: 2026-07-10T16:23:23Z

Description: Firebase Hosting / App-Distribution deployment plan for all Helix OTA surfaces + git-ignored secrets and fresh-clone manifest (operator-requested 2026-07-10; four-format export per §11.4.65).

Authority: Helix OTA deployment — docs/firebase.

Author: (T1/main - claude1) background research subagent **Scope:** Read-only investigation. No repo mutation, no git/build command executed, no live Firebase project created, nothing deployed. All claims below are either (a) a direct observation of a real file/path in this checkout, cited by path, or (b) a cited live web source retrieved 2026-07-10, or (c) explicitly marked PENDING when a live fetch did not return the needed detail. No value of any credential/token/key appears anywhere in this document — only variable **names** and file **paths**.

0. CRITICAL FINDINGS (read this first)

No tracked secret VALUE leak was found. The repo-wide audit (Task D, §4 below) found:

1. **No real leak.** Every literal-looking credential assignment found in tracked files resolved to one of: (a) a shell-required-variable pattern with no default (`${HELIX_ADMIN_PASSWORD:?Set HELIX_ADMIN_PASSWORD...}` in `clients/ota-manager/docker/ota-manager.docker-compose.yml:53`), (b) a widely-reused, clearly non-secret **test fixture** string also hard-coded in `server/internal/`

deviceemu/emulator_test.go:42 (fxAdminPass = "s3cret") and server/tests/chaos/chaos_test.go:71/85/142 — the *same* 6-char test password appears in .github/workflows/ci.yml.disabled-local-only:147,166 (a **disabled** CI workflow per §11.4.156 — filename literally ends .disabled-local-only), (c) an explicitly non-secret placeholder (HELIX_TOKEN_SECRET=ephemeral-test-stack-token-NOT-A-SECRET in docs/qa/20260623-chaos-live/boot.log:42), or (d) a **public** ed25519 verification key (HELIX_ARTIFACT_PUBKEY=jc0sQsDpso3JYXBYfx06eLkde04dx7Z390uLAKUgM0A=, same log line — public keys are not secrets by design, see §4.4).

2. **Evidence a prior leak-audit already worked correctly.** docs/qa/20260608-perf/run_conditions.txt:7 shows HELIX_TOKEN_SECRET=<redacted-§11.4.10> — the exact redaction marker §11.4.10.A mandates. Whatever real value was originally captured there was already found and scrubbed by an earlier audit. This is a positive signal, not a new finding.
3. **Operational gap, not a leak:** the host-level ~/api_keys.sh exports a variable named **FIRBASE_CLI_TOKEN** (note the typo — missing the second E in “FIREBASE”). Nothing in the tracked codebase currently reads this variable (grep -rI "FIRBASE_CLI_TOKEN\FIREBASE_TOKEN" over tracked *.sh/*.md returns empty) — there is no Firebase deploy script yet, so the typo has caused no functional breakage to date, but **any deploy script written against this var MUST use the exact existing (typo'd) name FIRBASE_CLI_TOKEN, or the operator must rename the export in ~/api_keys.sh first** (out of scope for this read-only task — flagging for operator decision per §11.4.66, not silently “fixing” a name I don’t own). Also see §4 — this token mechanism (firebase login:ci / FIREBASE_TOKEN) is itself **deprecated upstream** (§2.4), so wiring a new deploy script around it is a dead end regardless of the typo.
4. One tracked *.pem file exists — tests/emulator/ab_virt/rootfs-overlay/etc/rauc/dev.cert.pem. Confirmed by content inspection to be a **public X.509 certificate** (-----BEGIN CERTIFICATE-----), not a private key. Its private counterpart (dev.key.pem) is generated at build time by tests/emulator/ab_virt/rauc/gen_dev_keys.sh into the gitignored tests/emulator/ab_virt/out/rauc-keys/ tree and is **never tracked** (verified: git ls-files shows no dev.key.pem). This is an intentional, documented dev-only RAUC signing setup (unrelated to Firebase); flagged in §4 for completeness since the task asked for **all** sensitive/git-ignored data, not only Firebase-related items.

Net verdict: PASS — no credential value is currently committed to git in this repository.

1. Task A — Firebase token/CLI premise (observed facts, names only)

Check	Result
<code>~/api_keys.sh</code> exists	YES — it is a symlink: <code>/home/milos/api_keys.sh -> ./Factory/api_keys.sh</code> (resolves to <code>/home/milos/Factory/api_keys.sh</code>).
FIREBASE-related exported var names in <code>~/api_keys.sh</code>	Exactly one: FIRBASE_CLI_TOKEN (typo preserved verbatim from the file — this is the real exported name, not a transcription error on my part). No <code>FIREBASE_TOKEN</code> , no <code>GOOGLE_APPLICATION_CREDENTIALS</code> , no <code>FIREBASE_PROJECT_ID</code> are exported.
Other exported vars present (context only, names only)	~50 other provider API keys (<code>HUGGINGFACE_API_KEY</code> , <code>GEMINI_API_KEY</code> , <code>DEEPSEEK_API_KEY</code> , <code>GITLAB_TOKEN</code> , <code>GITFLIC_TOKEN</code> , <code>GITVERSE_TOKEN</code> , etc.) — none Firebase-specific besides the one above.
firebase CLI on PATH	YES — resolved to <code>/home/milos/Factory/software/Firebase/bin/firebase</code> .
<code>firebase --version</code>	15.22.3

Conclusion: the CLI is installed and current-looking; the token variable exists but (a) is misnamed relative to the CLI's own conventional `FIREBASE_TOKEN` env var, and (b) the underlying `firebase login:ci` token mechanism it presumably holds is itself deprecated upstream (§2.4) — so before any automation is written, the operator should decide between (i) renaming/re-exporting a service-account-based credential instead (recommended, see §2.4), or (ii) keeping token-based `FIRBASE_CLI_TOKEN` for now as an interim manual/local-only deploy path.

2. Task B — Latest Firebase deploy workflow research (live-fetched 2026-07-10)

2.1 Firebase Hosting deploy via CLI

Source: firebase.google.com/docs/hosting/quickstart (fetched 2026-07-10; page self-reports “Last updated 2026-07-03 UTC”).

1. Install/update the Firebase CLI (already present locally at v15.22.3 — §1).
2. From the project (or the specific app’s) root: `firebase init hosting` — interactive prompts: select/create the Firebase project, specify the **public root directory** (Firebase’s default suggestion is `public`; for a Vite app this **MUST** be pointed at the build output directory — for Helix OTA that is `dashboard/dist/`, see §3), and choose single-page-app rewrite behavior (yes, for a React Router SPA like `dashboard/`).
3. This produces two config files in the directory `firebase init` was run from:
 - **firebase.json** — hosting config (public dir, rewrites, ignore patterns, headers).
 - **.firebaserc** — project aliasing (which Firebase project this checkout deploys to). Neither file currently exists anywhere in this repo (confirmed: `git ls-files | grep -iE 'firebase\.json|\.firebaserc'` → empty) — Firebase Hosting has **never** been initialized here.
4. Deploy: `firebase deploy --only hosting` uploads the public directory’s contents to Firebase’s global CDN and activates the release.
5. **Preview channels** (share a build at a temporary URL before promoting to the live channel): `firebase hosting:channel:deploy <CHANNEL_ID> [--expires <duration>]`, e.g. `firebase hosting:channel:deploy new-awesome-feature --expires 7d`. --expires accepts h/d/w units, max 30 days. Confirmed via firebase.google.com/docs/hosting/manage-hosting-resources and corroborating GitHub Actions integration docs (firebase.google.com/docs/hosting/github-integration), web-searched 2026-07-10.

This is the correct path for “deploy the website so we can see & test it” for a **static SPA build** — exactly what `dashboard/` (and, separately, `clients/ota-manager/`’s `web/hostrender` build target) produce.

2.2 Firebase Hosting vs Firebase App Distribution

Source: firebase.google.com/docs/app-distribution (fetched 2026-07-10).

- **Firebase Hosting** = production/preview delivery of a **static web app** (HTML/JS/CSS bundle) to end users via a CDN + custom domain. This is what a browser-based dashboard needs.
- **Firebase App Distribution** = pre-release **binary** distribution to a private list of testers for **iOS and Android only** (“It does not support desktop or web applications” — direct reading of the official page). Distribution methods: Firebase console, Firebase CLI (`firebase appdistribution:distribute <path-to-binary> --app <APP_ID> --groups <tester-group>` is the documented pattern across Firebase’s own guides and quickstarts; the exact flag set was not fully extractable from this fetch — **PENDING** exact flag confirmation via `firebase appdistribution:distribute --help` at execution time), Gradle plugin (Android), or fastlane.
- **Direct implication for Helix OTA:** App Distribution covers the Android agent (`submodules/ota-android-agent`, an APK/AAB) but explicitly does **not** cover `clients/ota-manager/` (a **Tauri v2 desktop app** — Linux/macOS/Windows native binaries, see §3) or `dashboard/` (a browser SPA). The operator’s two phrases — “Firebase Distribute” and “Web app” — map to two **different, non-overlapping** Firebase products, and the desktop Tauri client maps to **neither** (a Windows/macOS/Linux installer needs a different distribution channel — e.g. GitHub Releases, a dedicated download page served by the very same Firebase Hosting site, or a future desktop-update mechanism — that decision is out of scope here and is flagged as an open question for the operator, not decided unilaterally per §11.4.66).

2.3 Firebase project + multi-platform app registration

Source: firebase.google.com/docs/projects/learn-more (fetched 2026-07-10), corroborated by web search 2026-07-10.

- A single **Firebase project** is the top-level container; every app registered inside it (Web / Android / iOS — called “platform variants”) **shares** the same backend resources/services. Firebase projects are Google Cloud projects underneath.
- Project cap: **30 registered apps per project** (per the official “learn more” page).
- CLI: `firebase projects:create --display-name "<name>"` creates the project (an interactive fallback with prompts also exists). `firebase`

`apps:create` creates an app inside the current project; multiple independent sources (search results, firebase `apps:sdconfig` WEB usage in a firebase-tools GitHub issue) confirm the CLI takes a **PLATFORM** argument whose values are WEB / ANDROID / IOS (e.g. the conventional form is `firebase apps:create <PLATFORM> "<display name>"`) — the **exact** full flag syntax (e.g. Android package name / iOS bundle ID flags) was not extractable from the fetched pages; this is marked **PENDING exact-syntax confirmation via firebase apps:create --help** rather than guessed, per §11.4.6.

- **Where the Go control-plane server fits:** Firebase does not host an arbitrary backend binary (no facility to run a long-lived Go process). Three honest options, none executed here: (a) register the server merely as a **Web app entry** purely so its API can be referenced/keyed from Firebase project metadata (no hosting of the binary itself); (b) if the operator later wants Firebase-native backend hosting, that is **Cloud Run** (a Google Cloud product Firebase Hosting can rewrite requests to — `firebase.json` `"rewrites": [{ "source": "/api/**", "run": { "serviceId": "..."} }]`) — this was referenced generically by the fetched project docs (“Cloud Functions for Firebase” / underlying GCP integration) but a dedicated Cloud-Run-rewrite doc was not separately fetched in this session — **PENDING** if the operator wants this path researched in depth; (c) simplest and recommended for now: leave the Go server deployed exactly as it already is (own container/host, see `server/Dockerfile`, `clients/ota-manager/docker/ota-manager.docker-compose.yml`) and use Firebase **only** for the static dashboard/website surface — Firebase Hosting’s rewrites can still proxy `/api/**` requests to that externally-hosted server without needing Cloud Run at all (a plain `"destination"` rewrite is also supported by Firebase Hosting for arbitrary origins in some configurations — **PENDING** verification of that exact rewrite capability, flagged rather than asserted).

2.4 Non-interactive CI auth: FIREBASE_TOKEN vs service account

Sources: official firebase-tools maintainer discussion github.com/firebase/firebase-tools/discussions/6283 (fetched 2026-07-10) + corroborating web search 2026-07-10 (Medium guide, GitHub issues #10726, #5650, prisma/ecosystem-tests#3570).

- **firebase login:ci** (which mints a `FIREBASE_TOKEN`) is **officially deprecated** per firebase-tools’ own deprecation warning, quoted directly from the discussion: “*Authenticating with FIREBASE_TOKEN* is

deprecated and will be removed in a future major version of firebase-tools.” No specific removal version/date was stated in the sources found — it is a “future major version” without a committed date as of this research.

- **Recommended replacement:** a **Google Cloud service-account JSON key**, supplied via the standard Google auth-library environment variable `GOOGLE_APPLICATION_CREDENTIALS=/path/to/key.json`. The CLI auto-discovers this via the Google Auth Library — no `firebase login` step needed at all in CI once this is set.
- **Known rough edge (informational, not blocking):** at least one open `firebase-tools` issue (#10726) reports `firebase deploy` failing to pick up Workload-Identity-Federation-issued credentials via `GOOGLE_APPLICATION_CREDENTIALS` in some CI providers (GitLab OIDC) — a traditional downloaded service-account-key JSON file (not a WIF token) is the well-trodden path.
- **Recommendation for Helix OTA:** do **not** build new automation around `FIRBASE_CLI_TOKEN` (deprecated mechanism + currently-unused typo’d name). Instead, when the operator is ready to execute this plan, create a Google Cloud **service account** with the Firebase Hosting Admin role (console: IAM & Admin → Service Accounts → Create → download JSON key), store the key **outside git** (see §4), and export `GOOGLE_APPLICATION_CREDENTIALS` in any deploy script/CI runner. This is a **new** secret this plan will introduce — it does not exist yet anywhere in this checkout or in `~/api_keys.sh`.

3. Task C — Helix OTA → Firebase project structure PLAN (recommendation only, not executed)

3.1 Real surfaces found in this checkout (grounded in actual paths)

#	Surface	Real path	What it is (observed, not assumed)	Exists today?
1	Go control-plane server	<code>server/</code> (module <code>github.com/HelixDevelopment/helix_ota/server</code> ,	Gin modular-monolith REST API (<code>/api/v1</code>); <code>httptest</code> integration suite in	YES, built + tested.

#	Surface	Real path	What it is (observed, not assumed)	Exists today?
2	Operator Dashboard (web SPA)	server/cmd/ota-server) dashboard/	server/internal/api/. React 18 + TypeScript + Vite SPA, react-router-dom, talks only to /api/v1 via dashboard/src/api/client.ts. Per dashboard/README.md: “a buildable scaffold ... It typechecks and builds” (see dashboard/BUILD_EVIDENCE.txt). A production build already exists on disk at dashboard/dist/ (index.html + assets/, last built 2026-07-10). Design source of truth: docs/research/main_specs/1.0.0-mvp/dashboard/dashboard_design.md. A Tauri v2 desktop app (package.json: “Helix OTA Manager — Tauri v2 desktop client for managing OTA updates”) — React/Vite UI shell (src/) wrapped by a native Rust shell (src-tauri/) that builds native installers/binaries	YES — this already exists and already builds. (The task brief’s premise that the dashboard “does not exist yet” is not accurate for this checkout — see honest correction below.)
3	OTA Manager desktop client	clients/ota-manager/		YES, present with its own build/docker/visual-test tooling.

#	Surface	Real path	What it is (observed, not assumed)	Exists today?
			per OS. Also has its own Docker path (clients/ota-manager/docker/ota-manager.docker-compose.yml) that serves its web build via nginx as a plain SPA.	
4	Android OTA agent	submodules/ota-android-agent/	Kotlin/KMP device agent (poll/download/verify/apply/report); builds an Android APK/AAB.	YES (submodule present, Gradle project).
5	Android update_engine bridge	submodules/ota-update-engine-bridge/	Kotlin library wrapping AOSP update_engine/boot_control; consumed BY the agent (#4), not itself a distributable end-user app.	YES (submodule present).
6	CLI/dev tooling	server/cmd/applyport, server/cmd/ota-device-emu, server/tools/loadtest, cmd/build-stats, cmd/workable-items	Internal Go dev/test/ops tools — not end-user distributables.	YES, internal-only.
7	Multi-tenant web dashboard	<i>(no separate path found beyond #2)</i>	Track-2 focus label in config/multitrack/the-factory.yaml reads: “helix_ota web dashboard + website — multi-tenant super-admin/user sign-in + account-selection UI surfaces	IN PROGRESS on feature/accounts-web, extending #2.

#	Surface	Real path	What it is (observed, not assumed)	Exists today?
			(feature/accounts-web)” — i.e. multi-tenant super-admin/account-selection UI is active, in-progress work on the SAME dashboard / surface (#2) , tracked under docs/research/accounts, not a separate app.	

Honest correction of the task brief’s premise: the brief states “the (planned) multi-tenant web dashboard/website” and instructs “mark clearly anything that does not yet exist as PLANNED.” Per direct filesystem observation, **the dashboard is not merely planned — a buildable, typechecked, previously-built React/Vite SPA already exists at dashboard/**, and a second, separate multi-tenant/account-selection **feature branch** (feature/accounts-web) is actively extending it right now (§11.4.6 no-guessing: this is what the repo actually shows, not what I expected to find).

3.2 Recommended Firebase product mapping (one project, multiple registered apps)

Recommend **one Firebase project**, e.g. display name Helix OTA, holding:

Surface	Firebase product	App nickname (suggested)	Config/env needed
dashboard/ (operator web dashboard)	Firebase Hosting (+ optional preview channels per PR/feature branch)	n/a — Hosting sites aren’t “apps”; optionally also register a Web app (firebase apps:create WEB "Helix OTA Dashboard") purely to get a Firebase Web SDK config object if any Firebase client SDK feature (Analytics, Remote Config,	firebase.json (public: "dashboard/dist", SPA rewrite to index.html, optional / api/** rewrite to the externally-

Surface	Firebase product	App nickname (suggested)	Config/env needed
		etc.) is ever wanted client-side.	hosted Go server or a Cloud Run wrapper), .firebase (project alias). Build step npm run build inside dashboard/ MUST run before firebase deploy.
clients/ota-manager/ (Tauri desktop client)	Neither Hosting nor App Distribution directly covers a desktop installer (§2.2 — App Distribution is iOS/Android only). Two sub-options: (a) host the web-preview build of its UI (its own vite build output, distinct from the native Tauri binary) on the same Firebase Hosting site as a secondary path/subdomain purely for browser-based visual QA (this project's hostrender tooling already builds a web target for exactly this kind of visual testing — see clients/ota-manager/visual/);	n/a for the native binary path; optional Web app registration for (a).	Only relevant if option (a) is chosen: a second firebase.json hosting target/site within the same project.

Surface	Firebase product	App nickname (suggested)	Config/env needed
	(b) for the actual native installers (.msi/.dmg/.AppImage from tauri build), Firebase has no first-party product — recommend GitHub Releases or a plain download page served from the same Hosting site, decided by the operator.		
submodules/ ota- android- agent/ (Android agent APK/AAB)	Firebase App Distribution	Register an Android app (firebase apps:create ANDROID "Helix OTA Android Agent" with its package name/applicationId) so App Distribution can attach to it; then firebase appdistribution:distribute app-release.apk --app <ANDROID_APP_ID> --groups <testers>.	Android applicationId (from the submodule's Gradle config — not inspected in depth in this read-only pass), a configured tester group in the Firebase console.
submodules/ ota-update- engine- bridge/	None — it is a library, not a distributable app; no Firebase registration needed.	—	—
server/ (Go control plane)	No Firebase product hosts this directly (§2.3). Recommend: keep on its existing deploy path (server/Dockerfile, scripts/	n/a, or a Web-app registration purely for project-metadata bookkeeping if desired.	None beyond what already exists (HELIX_* env vars, §4).

Surface	Firestore product	App nickname (suggested)	Config/env needed
	distribute_stack.sh → rootless podman on thinker.local/ amber.local per .env.example); Firestore Hosting's / api/** rewrite can point at it as an external origin if/ when that rewrite capability is confirmed (§2.3, marked PENDING).		

3.3 HARD GATE — explicit statement per task instructions

The website (dashboard/) Hosting deploy described in §2.1/§3.2 is **technically ready-to-execute right now** — a production build already exists at dashboard/dist/ and the app “typechecks and builds” per its own BUILD_EVIDENCE.txt. **However, per this task’s explicit constraints, nothing was deployed, no live Firestore project was created, and firestore init / firestore deploy were NOT run.** This document is the ready-to-execute recipe (§2.1 + §3.2), not a completed deployment. Before actually running it, the operator should also weigh the honest open items flagged as PENDING in §2.1/§2.3 (exact apps:create flag syntax, and whether a Firestore Hosting rewrite can target an arbitrary external origin vs. only Cloud Run) — neither blocks *creating* the Firestore project + Hosting site, only the optional API-proxy rewrite step.

4. Task D — Sensitive / git-ignored data + fresh-clone manifest

4.1 .gitignore sensitive-pattern coverage

Root .gitignore (/home/milos/Factory/projects/tools_and_research/helix_ota/.gitignore) — relevant sensitive/secret-adjacent lines (paraphrased, full file is 1 path, not reproduced verbatim beyond what's needed): - .env — the real environment file (git-ignored; .env.example is the tracked template). - .git-backups/, .git-backup-filter-repo/ — repo-op safety backups (§9), not secrets per se. - Various build-artifact/cache excludes (node_modules/, package-lock.json, *.db-wal/*.db-shm, qa-results/, etc.) — not credential-bearing, listed here only because the task asked for the full enumeration; none of these carry secrets.

28 submodule/subdirectory .gitignore files exist (found via `find . -name .gitignore`): containers/, docs_chain/, constitution/, .remember/, dashboard/, .codegraph/, tools/device_claim/, cmd/workable-items/, cmd/build-stats/, plus one per owned submodule (submodules/ota-rollout-engine, submodules/containers, submodules/vision_engine, submodules/llm_orchestrator, submodules/ota-telemetry-schema, submodules/challenges, submodules/ota-artifact-validator, submodules/ota-protocol, submodules/doc_processor, submodules/http3, submodules/helixqa, submodules/security, submodules/llm_provider, submodules/llms_verifier, submodules/ota-update-engine-bridge, submodules/ota-android-agent) plus tests/e2e/.gitignore, tests/security/.gitignore. dashboard/.gitignore specifically excludes /.e2e-artifact-key.json — “Ephemeral ed25519 artifact-signing key for the e2e harness — regenerated every run by e2e/global-setup.ts, NEVER committed” (direct quote from the file).

4.2 .env / .env.example — required keys for a fresh clone

.env.example (tracked, 109 lines) documents these optional overrides — **none are secrets**, they are deployment-topology knobs:

Var	Purpose	Sensitive?
HELIX_RELEASE_PREFIX	Release-tag / recording-filename prefix (§11.4.151/ §11.4.155).	No.

Var	Purpose	Sensitive?
HELIXTRACK_REMOTE_HOST, HELIXTRACK_REMOTE_USER	SSH targets for scripts/ distribute_stack.sh (rootless podman remote deploy).	Host/user names only — not a credential (auth is via pre- configured SSH key, not a password in this file).
HELIX_DISTRIBUTE_ON_COMMIT, HELIX_DIST_REMOTE_DIR	Deploy-automation toggles.	No.
HELIX_MAX_INFLIGHT	Server request- concurrency ceiling.	No.
HELIX_TRUST_TLS_PROXY	Operator assertion the process sits behind a trusted TLS-terminating proxy.	No.

Gap found (documentation, not a leak): the Go server's `server/internal/config/config.go` reads several env vars that **.env.example** **does NOT mention at all**: `HELIX_PORT`, `HELIX_API_BASE_PATH`, `HELIX_ARTIFACT_BASE_URL`, `HELIX_POLL_INTERVAL`, `HELIX_POLL_JITTER`, `HELIX_ACCESS_TOKEN_TTL`, `HELIX_DEVICE_TOKEN_TTL`, `HELIX_TLS_CERT`, `HELIX_TLS_KEY`, `HELIX_HTTPS_PORT`, `HELIX_DATABASE_URL`, and — the two that actually matter for secrets — **`HELIX_TOKEN_SECRET`** (symmetric JWT-signing secret; falls back to a hard-coded **non-production** dev default "helix-ota-dev-token-secret-change-me" if unset — see `config.go:183`) and **`HELIX_ARTIFACT_PUBKEY`** (base64 ed25519 **public** key trusted for artifact-signature verification; `server/internal/api/handlers_artifact.go:274-288`, `resolvePublicKey()` — per this project's CLAUDE.md trust-boundary rule, this key **MUST** come only from server config, never from a request). A fresh clone/deploy **MUST** set **`HELIX_TOKEN_SECRET`** to a real random secret in production (the checked-in dev fallback is explicitly labeled "change-me" and must not be used in production), and **MUST** set **`HELIX_ARTIFACT_PUBKEY`** to the base64 form of whatever ed25519 keypair signs real OTA artifacts (the corresponding **private** signing key is an operator-held secret that lives **entirely outside this repo** — no private artifact-signing key exists anywhere in this checkout, tracked or otherwise; only the *public* verification half is ever configured server-side).

4.3 ~/api_keys.sh (host-level, outside the repo) — names only

Already covered in §1: single Firebase-relevant name is **FIRBASE_CLI_TOKEN** (typo preserved). ~50 other provider-API-key names are present for unrelated tooling (LLM providers, git-forge tokens) — out of scope for this Firebase-focused audit beyond noting they exist and are correctly kept out of git (the file itself, /home/milos/Factory/api_keys.sh, is outside this repository entirely, not merely gitignored within it).

4.4 Artifact-signature keys (trust boundary)

Item	Path / source	Nature	Fresh-clone requirement
Trusted public verification key	server/internal/config/config.go reads HELIX_ARTIFACT_PUBKEY (base64 ed25519, 32 bytes) → Config.ArtifactPublicKey → Server.pubKey → resolvePublicKey() in server/internal/api/handlers_artifact.go	Not secret by cryptographic design (a verification public key), but MUST come only from trusted server config per this project's CLAUDE.md hard rule — never accept one from a request.	Operator must supply the real base64 public key value via HELIX_ARTIFACT_PUBKEY env var in production; unset = uploads are rejected (no trusted key configured) — this is a fail-closed default, confirmed safe.
Private signing key (signs real OTA artifacts before upload)	Not present anywhere in this checkout.	Secret.	Operator must supply/hold this entirely outside the repo (the build pipeline that signs release artifacts is the only consumer) — nothing to regenerate here since this repo contains no signing-key generation tooling for <i>production</i> artifacts

Item	Path / source	Nature	Fresh-clone requirement
JWT symmetric secret	HELIX_TOKEN_SECRET env var, config.go:180-184	Secret. Dev fallback "helix-ota-dev-token-secret-change-me" is intentionally weak/labeled.	(only the emulator's own throwaway RAUC dev key, §4.5, which is unrelated). Operator MUST set a real random value in any non-dev deployment.

4.5 RAUC dev signing keypair (emulator test harness — not Firebase-related, included for completeness per “audit ALL sensitive data”)

Item	Path	Nature	Fresh-clone requirement
Dev private key	Generated by tests/emulator/ab_virt/rauc/gen_dev_keys.sh into tests/emulator/ab_virt/out/rauc-keys/dev.key.pem (gitignored, chmod 600, never committed per the script's own header comment).	Secret (throwaway/dev-only).	Regenerable: run the script; §11.4.77 regen-mechanism already documented in the script itself. Not needed unless running the RK3588 A/B-virt emulator tests.
Dev public cert	Tracked: tests/emulator/ab_virt/rootfs-overlay/etc/	Public X.509 cert — confirmed safe to track (verified file starts -----	None — already in the tree.

Item	Path	Nature	Fresh-clone requirement
	rauc/ dev.cert.pem	BEGIN CERTIFICATE-----).	

4.6 Multitrack per-alias auth (CLAUDE_CODE_OAUTH_TOKEN / CLAUDE_CONFIG_DIR)

Referenced (names only, no values) throughout constitution/scripts/multitrack/*.sh (multitrack_sessions.sh, multitrack_cwd_hook.sh, multitrack_conductor_monitor_autoarm.sh, track_label_audit.sh). Per config/multitrack/the-factory.yaml's own comment: "*Claude Toolkit alias roster. NAMES ONLY — never keys/tokens (§11.4.10; keys live in host ~/api_keys.sh outside the repo).*" The tracked YAML lists alias **names** (claude1..claude4, deepseek, xiaomi, opencode, kimi-for-coding) and their kind (native/provider) only — confirmed no token values appear in this tracked file. A fresh clone that wants multi-track orchestration needs the operator to populate the corresponding CLAUDE_CODE_OAUTH_TOKEN/CLAUDE_CONFIG_DIR values per-alias on the host, outside git, exactly as the existing convention already requires — this is pre-existing project convention, not something this Firebase task needs to change.

4.7 Verification that no secret VALUE is tracked (methodology)

All of the following ran against `git ls-files / git grep` (tracked content only, not the working tree) and returned **empty** (no hits) except where explicitly noted and resolved above: - `.env` is not tracked (only `.env.example` is). - No `.env.*` besides `.env.example` is tracked. - No `firebase.json` / `.firebaserc` tracked anywhere (Firebase Hosting has never been initialized in this repo). - No PEM/private-key header (BEGIN RSA PRIVATE KEY / BEGIN PRIVATE KEY / BEGIN OPENSSH PRIVATE KEY / BEGIN EC PRIVATE KEY) appears in any tracked file. - No Google API key pattern (`AIza[0-9A-Za-z_-]{35}`) appears in any tracked file. - The one tracked `*.pem/*.key`-adjacent filename hit (`tests/emulator/ab_virt/rootfs-overlay/etc/rauc/dev.cert.pem`) is a public certificate (§4.5). - Every literal `HELIX_ADMIN_PASSWORD=/HELIX_TOKEN_SECRET=/` `HELIX_ARTIFACT_PUBKEY=` assignment found in tracked files was individually inspected and resolved to a non-secret/test-fixture/already-redacted/public-key value (§0, §4.2).

5. Summary table — fresh-clone secrets checklist

#	What	Path (repo-relative unless noted)	Why ignored	What a fresh clone must supply
1	Real environment file	<code>.env</code> (repo root)	§11.4.10 — never commit real config/secrets	Copy <code>.env.example</code> → <code>.env</code> in real values
2	JWT signing secret	env var <code>HELIX_TOKEN_SECRET</code>	Secret material	Random 32+ byte value in production
3	Artifact verification public key	env var <code>HELIX_ARTIFACT_PUBKEY</code>	Trust-boundary config (not secret, but must come from config)	Base64 ed25519 public key matching the real signing pipeline
4	Artifact signing private key	<i>(not in this repo at all)</i>	Secret, never touches the server repo	The build pipeline that signs release artifacts must hold it
5	Admin bootstrap password	env var <code>HELIX_ADMIN_PASSWORD</code>	Secret	Strong password (docker-compose enforces “no de via <code>:?</code> syntax)
6	Firebase deploy credential	<i>(does not exist in usable form yet)</i> — <code>~/api_keys.sh</code> ’s <code>FIRBASE_CLI_TOKEN</code> is unused/deprecated-mechanism	Would be a deploy secret	A service-account JSON key path via <code>GOOGLE_APPLICATION_CREDENTIALS</code> (recommended, §2.4)
7	RAUC dev signing private key	<code>tests/emulator/ab_virt/out/rauc-keys/dev.key.pem</code> (gitignored)	Dev/test-only secret	Only needed for RK3588 virt emulator tests
8	E2E ephemeral	<code>dashboard/.e2e-artifact-key.json</code> (gitignored)	Ephemeral test secret	Only needed to run dashboard e2e suite

#	What	Path (repo-relative unless noted)	Why ignored	What a fresh clone must supply
	artifact-signing key			
9	Per-alias Claude Code auth	CLAUDE_CODE_OAUTH_TOKEN / CLAUDE_CONFIG_DIR (host env, per alias)	§11.4.10	Only needed for multi-trace orchestration
10	Distribution-host SSH access	implied by HELIXTRACK_REMOTE_HOST/_USER in .env.example	Not itself a secret in-repo; relies on pre-provisioned SSH key auth	Passwordless SSH key for configured user on the target host(s)

6. Anti-bluff / retrieval notes

- All Task B claims are cited with the exact URL fetched and the retrieval date (2026-07-10, matching the environment's current date). Two sub-items could not be fully confirmed from the fetched page content within this session and are explicitly marked **PENDING** rather than guessed: (a) the complete flag syntax for `firebase apps:create <PLATFORM> / firebase appdistribution:distribute`, (b) whether a Firebase Hosting rewrite can target an arbitrary external HTTP origin vs. only Cloud Run/Cloud Functions. Both are easy to resolve at execution time with `firebase <command> --help` or a follow-up official-docs fetch — flagging honestly per §11.4.6/§11.4.99 rather than asserting an unverified syntax.
- All Task C/D claims are grounded in files actually read in this checkout during this session; paths are given exactly as found. Where the task brief's assumption ("dashboard doesn't exist yet") conflicted with what the repository actually contains, this document states the observed reality rather than the assumed premise (§3.1).
- No repo file was modified, no git/build command was executed, no Firebase project was created, and nothing was deployed, per the task's explicit constraints.